

Business Logic Flaws & AuthZ Gaps

Why scanners miss logic flaws

A scanner sends mutated payloads. A logic flaw requires reading the *feature*: pricing, gift cards, state transitions, permissions – the app's own business rules become the exploit.

Flavor menu

Flaw	Pattern
Price tampering	Change price/qty/currency in request body
Coupon abuse	Reuse once-coupons; negative quantities; race
Balance manipulation	Withdraw → deposit race (double-spend)
State-skip	Jump from 'cart' straight to 'shipped' by calling admin API
Race conditions	Two requests win one credit (TOCTOU)
Rate-limit evasion	Rotate User-Agent, IP, or header tricks on login
Referral/fraud	Self-referral loops, fake KYC onboarding

The manual test rhythm

Read the flow: what transitions are allowed? What's enforced client-side only?

Tamper the request you can control (Repeater): price field, step number, role, quantity

Run the same action twice quickly (race) – the DB writes twice

Look for 'hidden' endpoints the frontend uses but UI hides

```
# classic spot:
POST /cart/checkout {"price":10}    # set price server-side!
GET  /api/order/state                # can you PATCH step from 'review' to 'complete'?
POST /vouchers/redeem {"code":"FREEDEL"} # twice in a row?
```

AuthZ gaps (IDOR/BOLA reprise, deepened)

Object-level: any ID works (neighbors)

Batch: bulk endpoints with mixed permissions

Missing hierarchy: user vs admin role toggled only by cookie flag

Brain: object ACL enforced at a different layer (UI, not API)

Fix culture

Server-side state machine for transitions (never client hints)

Price/qty/role fields server-derived, user hints rejected

Idempotency keys for money-moving endpoints (race killer)

Central authz middleware (single enforcement point, tested)

Boundary/write-audit tests for money + privilege features

Logic flaws are the AppSec 'intelligence' exam: 95% infrastructure, 5% privilege. The privilege part wins money and data.

[← JWT Attacks & API Security](#)

[From Attack to Fix: Secure Web...](#)